

Appian SLA Predictive Simulation

Process Mining and Predictive Analytics Platform



Documentation Version 1.0

December 2025

Abstract

This document provides comprehensive documentation for the Appian SLA Predictive Simulation platform, a sophisticated process mining and predictive analytics system designed for Appian workflows. The platform leverages machine learning to predict activity durations, next activities, and SLA breach risks with real-time event streaming capabilities. It integrates multiple technologies including Apache Kafka for event streaming, ClickHouse for analytics storage, XGBoost for machine learning predictions, and provides an interactive web-based dashboard for monitoring and what-if scenario analysis.

Contents

1	Introduction	8
1.1	Overview	8
1.2	Key Objectives	8
1.3	Target Audience	8
2	System Architecture	8
2.1	High-Level Architecture	8
2.2	Component Description	9
2.2.1	Infrastructure Components	9
2.2.2	Application Services	9
2.3	Data Flow Architecture	10
3	Technology Stack	10
3.1	Backend Technologies	10
3.2	Frontend Technologies	10
4	Machine Learning Models	11
4.1	Model Overview	11
4.2	Model Architecture	11
4.3	Feature Engineering	11
4.3.1	Duration Model Features	11
4.3.2	SLA Breach Model Features	11
5	Core Services	12
5.1	Data Simulator Service	12
5.1.1	Purpose	12
5.1.2	Key Features	12
5.1.3	API Endpoints	12
5.1.4	Usage Example	12
5.2	SLA Monitor Service	12
5.2.1	Purpose	12
5.2.2	Key Features	13
5.2.3	API Endpoints	13
5.3	Inference Service	13
5.3.1	Purpose	13
5.3.2	Key Features	13
5.3.3	API Endpoints	14
5.4	Simulation Sandbox Service	14
5.4.1	Purpose	14
5.4.2	Key Features	14
5.4.3	API Endpoints	14
6	Database Schema	14
6.1	ClickHouse Events Table	14
6.2	Common Queries	15
6.2.1	Total Events Count	15
6.2.2	Active Cases	15

6.2.3	Queue Depths by Activity	15
6.2.4	SLA Breaches	16
7	Demo UI	16
7.1	Overview	16
7.2	Features	16
7.2.1	Live Dashboard	16
7.2.2	Manual Predictions	16
7.2.3	What-If Sandbox	16
7.2.4	Database Queries	17
7.3	Starting the Demo UI	17
8	Installation and Deployment	17
8.1	Prerequisites	17
8.2	Quick Start Guide	17
8.2.1	Step 1: Clone Repository	17
8.2.2	Step 2: Start All Services	17
8.2.3	Step 3: Verify Services	18
8.2.4	Step 4: Generate Test Data	18
8.2.5	Step 5: Verify Data Flow	18
8.2.6	Step 6: Access Demo UI	18
8.3	Service Health Checks	18
9	Configuration	18
9.1	Environment Variables	18
9.2	Process Configuration	19
9.3	Model Registry	19
10	Monitoring and Observability	19
10.1	Prometheus Metrics	19
10.2	Grafana Dashboards	20
11	Troubleshooting	20
11.1	Common Issues	20
11.1.1	Services Not Starting	20
11.1.2	No Data in ClickHouse	20
11.1.3	Kafka Connection Issues	21
11.2	Log Analysis	21
12	Development Guide	21
12.1	Project Structure	21
12.2	Development Workflow	22
12.2.1	Rebuild Specific Service	22
12.2.2	View Real-Time Logs	22
12.2.3	Stop Services	22
12.3	Adding New Models	22
13	Use Cases	22
13.1	Proactive SLA Management	22

13.2 Process Optimization	23
13.3 Capacity Planning	23
13.4 Process Mining	23
14 Security Considerations	24
14.1 Development vs Production	24
14.2 Data Privacy	24
15 Performance Optimization	24
15.1 ClickHouse Optimization	24
15.2 Kafka Optimization	24
15.3 Model Inference Optimization	25
16 API Reference	25
16.1 Data Simulator API	25
16.1.1 POST /api/generate	25
16.2 Inference API	25
16.2.1 POST /predict/duration	25
16.2.2 POST /predict/sla_breach	26
16.3 SLA Monitor API	26
16.3.1 GET /state/wip	26
16.3.2 GET /bottlenecks/current	26
16.4 Simulation Sandbox API	27
16.4.1 POST /scenario/create	27
16.4.2 POST /scenario/compare	27
17 Future Enhancements	28
17.1 Planned Features	28
17.2 Research Directions	28
18 Conclusion	28
19 References	29
A Appendix A: Sample Data	29
A.1 Sample Event Record	29
B Appendix B: Configuration Files	29
B.1 Model Registry Example	29
C Appendix C: Glossary	30

List of Figures

1	System Architecture Overview	9
---	--	---

List of Tables

1	Infrastructure Components	9
2	Application Services	9
3	Backend Technology Stack	10
4	Frontend Technology Stack	10
5	Machine Learning Models	11
6	Data Simulator API Endpoints	12
7	SLA Monitor API Endpoints	13
8	Inference Service API Endpoints	14
9	Simulation Sandbox API Endpoints	14
10	Environment Variables	19
11	Prometheus Metrics	19

1 Introduction

1.1 Overview

The Appian SLA Predictive Simulation platform is an advanced process mining and predictive analytics solution designed to optimize business process management in Appian workflows. The system provides real-time monitoring, predictive analytics, and simulation capabilities to help organizations proactively manage Service Level Agreements (SLAs) and optimize resource allocation.

1.2 Key Objectives

The platform addresses several critical business needs:

- **Proactive SLA Management:** Predict and prevent SLA breaches before they occur
- **Process Optimization:** Identify bottlenecks and inefficiencies in real-time
- **Capacity Planning:** Simulate different resource allocation scenarios
- **Data-Driven Decisions:** Leverage machine learning for accurate predictions
- **Real-Time Monitoring:** Track process execution and performance metrics continuously

1.3 Target Audience

This documentation is intended for:

- System Architects and Developers
- DevOps Engineers
- Business Process Analysts
- Data Scientists
- IT Operations Teams

2 System Architecture

2.1 High-Level Architecture

The system follows a microservices architecture pattern with event-driven communication. Figure 1 illustrates the main components and their interactions.

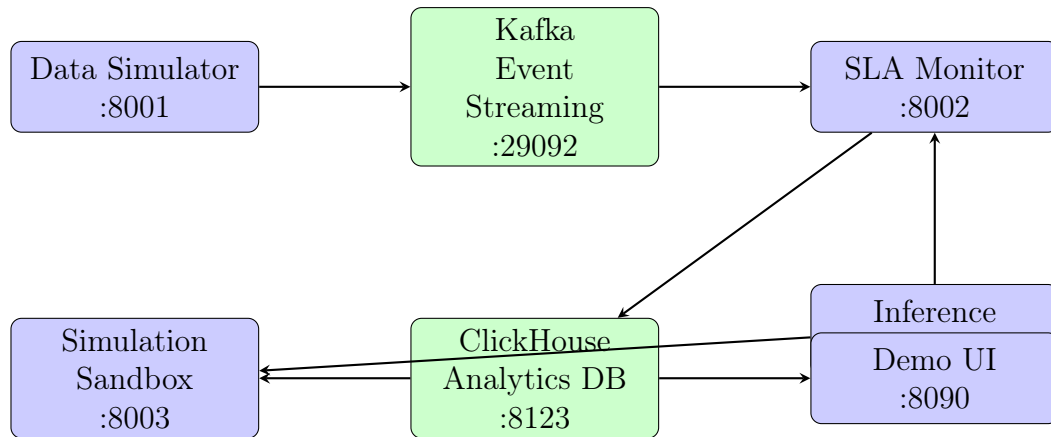


Figure 1: System Architecture Overview

2.2 Component Description

2.2.1 Infrastructure Components

Component	Description
Apache Kafka	Distributed event streaming platform for real-time data pipelines
ClickHouse	Column-oriented OLAP database for analytics and event storage
Zookeeper	Coordination service for Kafka cluster management
Prometheus	Metrics collection and monitoring system
Grafana	Visualization and analytics platform
Apache Flink	Stream processing framework for complex event processing

Table 1: Infrastructure Components

2.2.2 Application Services

Service	Port	Description
Data Simulator	8001	Generates realistic loan application process events
SLA Monitor	8002	Real-time SLA breach prediction and bottleneck detection
Simulation Sandbox	8003	What-if scenario testing and comparison
Inference Service	8004	ML model serving using XGBoost
Demo UI	8090	Interactive web dashboard with query executor

Table 2: Application Services

2.3 Data Flow Architecture

The data flows through the system in the following sequence:

1. **Event Generation:** Data Simulator generates realistic process events
2. **Event Streaming:** Events are published to Kafka topic "appian-events"
3. **Event Consumption:** SLA Monitor consumes events via background thread
4. **Event Storage:** Events are persisted in ClickHouse database
5. **State Management:** SLA Monitor queries ClickHouse for work-in-progress state
6. **ML Predictions:** Inference Service provides duration, routing, and breach predictions
7. **Analysis:** SLA Monitor forecasts breaches and detects bottlenecks
8. **Simulation:** Sandbox runs what-if scenarios using current state
9. **Visualization:** Demo UI displays results and allows interactive queries

3 Technology Stack

3.1 Backend Technologies

Technology	Version	Purpose
Python	3.9+	Primary programming language
FastAPI	Latest	RESTful API framework
Apache Kafka	7.4.0	Event streaming
ClickHouse	23.8	Analytics database
XGBoost	Latest	Machine learning models
Docker	Latest	Containerization
Docker Compose	3.8	Service orchestration

Table 3: Backend Technology Stack

3.2 Frontend Technologies

Technology	Purpose
HTML5	Structure and semantic markup
CSS3	Styling and responsive design
JavaScript (Vanilla)	Interactive functionality
Python HTTP Server	Static file serving and API proxy

Table 4: Frontend Technology Stack

4 Machine Learning Models

4.1 Model Overview

The platform employs three XGBoost-based machine learning models for different prediction tasks:

Model	Purpose	Key Features
Duration Predictor	Estimates activity completion time	Activity type, customer tier, complexity score, priority
Routing Predictor	Predicts next activity in process	Current activity, case state, historical patterns
SLA Breach Predictor	Calculates breach probability	Hours to SLA, complexity, activity type, customer tier

Table 5: Machine Learning Models

4.2 Model Architecture

All models use XGBoost (eXtreme Gradient Boosting), a highly efficient and scalable implementation of gradient boosting framework. The models are:

- Trained on historical process execution data
- Stored in JSON format for portability
- Versioned in the `weights/xgboost/v1/` directory
- Registered in `weights/registry.yaml` for centralized management

4.3 Feature Engineering

4.3.1 Duration Model Features

```

1 {
2   "activity": "Manual Review",
3   "customer_tier": "Gold",
4   "complexity_score": 7.5,
5   "priority": 2
6 }
```

Listing 1: Duration Model Input Features

4.3.2 SLA Breach Model Features

```

1 {
2   "activity": "Manual Review",
3   "hours_to_sla": 4,
4   "complexity_score": 8,
5   "customer_tier": "Gold"
}
```

6 }

Listing 2: SLA Breach Model Input Features

5 Core Services

5.1 Data Simulator Service

5.1.1 Purpose

Generates realistic loan application process events with configurable parameters to simulate various business scenarios.

5.1.2 Key Features

- Realistic event generation with temporal patterns
- Simulates variability (peak hours, agent availability, customer tiers)
- Configurable case volume and complexity
- Publishes events to Kafka in real-time

5.1.3 API Endpoints

Endpoint	Method	Description
/api/generate	POST	Generate specified number of test cases
/health	GET	Service health check

Table 6: Data Simulator API Endpoints

5.1.4 Usage Example

```
1 curl -X POST http://localhost:8001/api/generate \
2   -H "Content-Type: application/json" \
3   -d '{"num_cases": 100}'
```

Listing 3: Generate Test Data

5.2 SLA Monitor Service

5.2.1 Purpose

Monitors process execution in real-time, predicts SLA breaches, and identifies bottlenecks.

5.2.2 Key Features

- Real-time event consumption from Kafka
- Work-in-progress (WIP) state management
- SLA breach forecasting with probability scores
- Bottleneck detection and alerting
- Prometheus metrics exposure

5.2.3 API Endpoints

Endpoint	Method	Description
/state/wip	GET	Current work-in-progress cases
/bottlenecks/current	GET	Active bottlenecks
/breaches/forecast	GET	SLA breach predictions
/alerts/active	GET	Active alerts
/metrics	GET	Prometheus metrics
/health	GET	Service health check

Table 7: SLA Monitor API Endpoints

5.3 Inference Service

5.3.1 Purpose

Serves machine learning models for duration prediction, routing prediction, and SLA breach probability calculation.

5.3.2 Key Features

- Model registry for centralized model management
- Feature preprocessing and validation
- Fast prediction serving using XGBoost
- Support for multiple model versions

5.3.3 API Endpoints

Endpoint	Method	Description
/models	GET	List available models
/predict/duration	POST	Predict activity duration
/predict/routing	POST	Predict next activity
/predict/sla_breach	POST	Predict breach probability
/health	GET	Service health check

Table 8: Inference Service API Endpoints

5.4 Simulation Sandbox Service

5.4.1 Purpose

Enables what-if scenario testing by simulating process execution under different resource allocation configurations.

5.4.2 Key Features

- State snapshot management
- Scenario creation and configuration
- Monte Carlo simulation for statistical analysis
- Baseline vs scenario comparison
- Impact analysis on SLA compliance and throughput

5.4.3 API Endpoints

Endpoint	Method	Description
/state/snapshot	GET	Current state snapshot
/scenario/create	POST	Create new scenario
/scenario/compare	POST	Run and compare scenarios
/health	GET	Service health check

Table 9: Simulation Sandbox API Endpoints

6 Database Schema

6.1 ClickHouse Events Table

The primary data storage uses ClickHouse’s MergeTree engine for efficient analytics queries.

```

1 CREATE TABLE events (
2     event_id UInt64,
3     case_id String,
4     activity String,
5     status String,
6     timestamp DateTime,
7     resource String,
8     sla_deadline DateTime,
9     priority UInt8,
10
11     -- Appian-compatible column aliases
12     c0 String ALIAS activity,
13     c1 String ALIAS status,
14     c2 DateTime ALIAS timestamp,
15     c3 DateTime ALIAS sla_deadline,
16     c4 UInt8 ALIAS priority,
17     c5 String ALIAS case_id
18 ) ENGINE = MergeTree()
19 ORDER BY (case_id, timestamp);

```

Listing 4: ClickHouse Events Table Schema

6.2 Common Queries

6.2.1 Total Events Count

```

1 SELECT count() as total_events FROM events;

```

Listing 5: Count Total Events

6.2.2 Active Cases

```

1 SELECT case_id, activity, status, timestamp
2 FROM events
3 WHERE case_id NOT IN (
4     SELECT case_id FROM events
5     WHERE activity IN ('Approve', 'Reject')
6     AND status = 'COMPLETED'
7 )
8 ORDER BY timestamp DESC
9 LIMIT 10;

```

Listing 6: Query Active Cases

6.2.3 Queue Depths by Activity

```

1 SELECT activity, count() as queue_depth
2 FROM events
3 WHERE status = 'ASSIGNED'
4 GROUP BY activity
5 ORDER BY queue_depth DESC;

```

Listing 7: Calculate Queue Depths

6.2.4 SLA Breaches

```
1 SELECT case_id, activity, timestamp, sla_deadline
2 FROM events
3 WHERE timestamp > sla_deadline;
```

Listing 8: Identify SLA Breaches

7 Demo UI

7.1 Overview

The Demo UI provides an interactive web-based dashboard for monitoring, querying, and analyzing the process execution data.

7.2 Features

7.2.1 Live Dashboard

- Real-time system metrics
- Active case count
- Events per minute
- Average breach risk
- Risk monitor table with auto-refresh

7.2.2 Manual Predictions

- Duration estimator with sample data auto-fill
- SLA risk analyzer
- Interactive form inputs
- Real-time prediction results

7.2.3 What-If Sandbox

- Scenario configuration builder
- Resource pool adjustments
- Impact analysis visualization
- Baseline vs scenario comparison charts

7.2.4 Database Queries

- Interactive ClickHouse SQL executor
- Pre-built sample queries
- Custom query support
- Results displayed in formatted tables
- Keyboard shortcut: Ctrl+Enter to execute

7.3 Starting the Demo UI

```
1 cd demo
2 python3 server.py
3 # Open http://localhost:8090 in browser
```

Listing 9: Start Demo UI Server

8 Installation and Deployment

8.1 Prerequisites

- Docker Engine 20.10+
- Docker Compose 3.8+
- Python 3.9+ (for local development)
- 8GB RAM minimum (16GB recommended)
- 20GB free disk space

8.2 Quick Start Guide

8.2.1 Step 1: Clone Repository

```
1 git clone <repository-url>
2 cd Appian_SLA_PS
```

Listing 10: Clone Repository

8.2.2 Step 2: Start All Services

```
1 docker compose up -d --build
```

Listing 11: Start Services

8.2.3 Step 3: Verify Services

```
1 docker compose ps
```

Listing 12: Check Service Status

8.2.4 Step 4: Generate Test Data

```
1 curl -X POST http://localhost:8001/api/generate \  
2 -H "Content-Type: application/json" \  
3 -d '{"num_cases": 50}'
```

Listing 13: Generate Test Events

8.2.5 Step 5: Verify Data Flow

```
1 # Wait 5-10 seconds for data to flow  
2 sleep 8  
3  
4 # Check event count  
5 docker exec clickhouse clickhouse-client \  
6 --query "SELECT count() FROM events"
```

Listing 14: Verify ClickHouse Data

8.2.6 Step 6: Access Demo UI

```
1 cd demo  
2 python3 server.py  
3 # Open http://localhost:8090
```

Listing 15: Start Demo UI

8.3 Service Health Checks

All services expose health check endpoints:

```
1 curl http://localhost:8001/health # Data Simulator  
2 curl http://localhost:8002/health # SLA Monitor  
3 curl http://localhost:8003/health # Simulation Sandbox  
4 curl http://localhost:8004/health # Inference Service
```

Listing 16: Health Check Commands

9 Configuration

9.1 Environment Variables

Key environment variables configured in `docker-compose.yml`:

Variable	Description
KAFKA_BOOTSTRAP_SERVERS	Kafka broker connection string (kafka:9093)
CLICKHOUSE_HOST	ClickHouse server hostname (clickhouse)
INFERENCE_URL	Inference service URL (http://inference:8004)
MODEL_REGISTRY_PATH	Path to model registry YAML file
PROCESS_CONFIG	Path to process definition YAML

Table 10: Environment Variables

9.2 Process Configuration

Process definitions are stored in `config/process/loan_process.yaml` and define:

- Activity definitions and transitions
- Resource pools and capacities
- SLA deadlines per activity
- Priority levels
- Customer tier configurations

9.3 Model Registry

The model registry (`weights/registry.yaml`) contains:

- Model metadata (name, version, type)
- Model file paths
- Feature specifications
- Input/output schemas

10 Monitoring and Observability

10.1 Prometheus Metrics

The SLA Monitor exposes Prometheus metrics at `/metrics`:

Metric	Description
queue_depth{activity}	Queue size per activity
resource_utilization{resource}	Utilization percentage per agent
sla_breach_probability{case_id}	Breach probability per case
events_processed_total	Total events processed counter
prediction_latency_seconds	ML prediction latency histogram

Table 11: Prometheus Metrics

10.2 Grafana Dashboards

Pre-configured Grafana dashboards (accessible at `http://localhost:3000`):

- Process Overview Dashboard
- SLA Compliance Dashboard
- Resource Utilization Dashboard
- System Performance Dashboard

Default credentials: `admin/admin`

11 Troubleshooting

11.1 Common Issues

11.1.1 Services Not Starting

```
1 # Check service logs
2 docker compose logs -f sla-monitor
3
4 # Restart specific service
5 docker compose restart sla-monitor
6
7 # Check container status
8 docker compose ps
```

Listing 17: Debug Service Issues

11.1.2 No Data in ClickHouse

```
1 # Check consumer connection
2 docker compose logs sla-monitor | grep "EventConsumer"
3
4 # Restart SLA Monitor
5 docker compose restart sla-monitor
6
7 # Generate fresh data
8 curl -X POST http://localhost:8001/api/generate \
9   -H "Content-Type: application/json" \
10   -d '{"num_cases": 50}'
11
12 # Verify after waiting
13 sleep 8
14 docker exec clickhouse clickhouse-client \
15   --query "SELECT count() FROM events"
```

Listing 18: Debug Data Flow

11.1.3 Kafka Connection Issues

```

1 # Check Kafka status
2 docker compose ps kafka
3
4 # List topics
5 docker exec kafka kafka-topics \
6   --list --bootstrap-server localhost:9093
7
8 # Check consumer group
9 docker exec kafka kafka-consumer-groups \
10  --bootstrap-server localhost:9093 \
11  --describe --group sla-monitor-consumer

```

Listing 19: Debug Kafka

11.2 Log Analysis

```

1 # View all logs
2 docker compose logs
3
4 # Follow specific service logs
5 docker compose logs -f sla-monitor
6
7 # View last 100 lines
8 docker compose logs --tail=100 inference

```

Listing 20: View Service Logs

12 Development Guide

12.1 Project Structure

```

1 .
2     services/
3         data_simulator/      # Event generation
4         sla_monitor/         # SLA monitoring
5         inference/           # ML serving
6         simulation_sandbox/   # Scenario testing
7     config/
8         clickhouse/          # DB initialization
9         kafka/               # Kafka config
10        process/              # Process definitions
11        prometheus/           # Metrics config
12        grafana/              # Dashboard config
13    weights/
14        xgboost/v1/           # Model files
15        registry.yaml         # Model registry
16    demo/
17        index.html            # Dashboard UI
18        server.py             # HTTP server
19        Dockerfile            # Container config
20    docker-compose.yml        # Orchestration

```

```
21 README.md # Documentation
```

Listing 21: Directory Structure

12.2 Development Workflow

12.2.1 Rebuild Specific Service

```
1 docker compose build sla-monitor
2 docker compose up -d --build sla-monitor
```

Listing 22: Rebuild Service

12.2.2 View Real-Time Logs

```
1 docker compose logs -f sla-monitor
```

Listing 23: Monitor Logs

12.2.3 Stop Services

```
1 # Stop all services
2 docker compose down
3
4 # Stop and remove volumes (clears data)
5 docker compose down -v
```

Listing 24: Stop Services

12.3 Adding New Models

1. Train model and export to JSON format
2. Place model file in `weights/xgboost/v1/`
3. Update `weights/registry.yaml` with model metadata
4. Add prediction endpoint in `services/inference/app/main.py`
5. Rebuild inference service

13 Use Cases

13.1 Proactive SLA Management

Scenario: A loan processing team needs to ensure 95% of applications are approved within 48 hours.

Solution:

1. Monitor active cases in real-time via SLA Monitor

2. Predict which cases will breach SLA using ML models
3. Receive proactive alerts for at-risk cases
4. Take corrective action (reassign resources, escalate priority)

13.2 Process Optimization

Scenario: Identify and eliminate bottlenecks in the approval process.

Solution:

1. Analyze queue depths by activity
2. Identify bottlenecks automatically
3. Use simulation sandbox to test resource changes
4. Measure impact before implementation

13.3 Capacity Planning

Scenario: Plan staffing for expected volume increase during peak season.

Solution:

1. Simulate volume spikes using data simulator
2. Test different staffing levels in sandbox
3. Analyze SLA compliance under various scenarios
4. Optimize resource distribution

13.4 Process Mining

Scenario: Discover inefficiencies and patterns in historical data.

Solution:

1. Query event logs in ClickHouse
2. Analyze process execution patterns
3. Identify deviations from expected flow
4. Generate insights for process improvement

14 Security Considerations

14.1 Development vs Production

Warning: This is a development/demo setup. For production deployment:

- **Authentication:** Add authentication to all service endpoints
- **TLS/SSL:** Use encrypted connections for all services
- **Kafka Security:** Implement SASL/SSL for Kafka
- **Access Controls:** Implement role-based access control (RBAC)
- **Secrets Management:** Use proper secrets management (HashiCorp Vault, AWS Secrets Manager)
- **Network Isolation:** Use private networks and firewalls
- **Input Validation:** Validate and sanitize all inputs
- **Rate Limiting:** Implement API rate limiting

14.2 Data Privacy

- Ensure compliance with data protection regulations (GDPR, CCPA)
- Implement data anonymization for sensitive information
- Use encryption at rest for ClickHouse data
- Implement audit logging for data access

15 Performance Optimization

15.1 ClickHouse Optimization

- Use appropriate partitioning strategy
- Optimize ORDER BY clause for query patterns
- Implement materialized views for common aggregations
- Configure proper memory limits

15.2 Kafka Optimization

- Adjust partition count based on throughput
- Configure appropriate retention policies
- Tune consumer group settings
- Monitor consumer lag

15.3 Model Inference Optimization

- Batch predictions when possible
- Cache frequently requested predictions
- Use model quantization for faster inference
- Monitor prediction latency

16 API Reference

16.1 Data Simulator API

16.1.1 POST /api/generate

Generate test process events.

Request Body:

```
1 {  
2   "num_cases": 100  
3 }
```

Response:

```
1 {  
2   "status": "success",  
3   "cases_generated": 100,  
4   "events_generated": 823,  
5   "timestamp": "2025-12-25T23:50:00Z"  
6 }
```

16.2 Inference API

16.2.1 POST /predict/duration

Predict activity duration.

Request Body:

```
1 {  
2   "activity": "Manual Review",  
3   "customer_tier": "Gold",  
4   "complexity_score": 7.5,  
5   "priority": 2  
6 }
```

Response:

```
1 {  
2   "predicted_duration_hours": 4.2,  
3   "confidence": 0.87,  
4   "model_version": "v1"  
5 }
```

16.2.2 POST /predict/sla_breach

Predict SLA breach probability.

Request Body:

```
1 {
2   "activity": "Manual Review",
3   "hours_to_sla": 4,
4   "complexity_score": 8,
5   "customer_tier": "Gold"
6 }
```

Response:

```
1 {
2   "breach_probability": 0.73,
3   "risk_level": "high",
4   "recommended_action": "escalate",
5   "model_version": "v1"
6 }
```

16.3 SLA Monitor API

16.3.1 GET /state/wip

Get current work-in-progress cases.

Response:

```
1 {
2   "wip_count": 45,
3   "cases": [
4     {
5       "case_id": "CASE_000123",
6       "current_activity": "Manual Review",
7       "status": "ASSIGNED",
8       "hours_to_sla": 6.5,
9       "breach_risk": 0.34
10    }
11  ]
12 }
```

16.3.2 GET /bottlenecks/current

Get current bottlenecks.

Response:

```
1 {
2   "bottlenecks": [
3     {
4       "activity": "Manual Review",
5       "queue_depth": 23,
6       "avg_wait_time_hours": 8.5,
7       "resource_utilization": 0.95,
8       "severity": "high"
9     }
10  ]
11 }
```

16.4 Simulation Sandbox API

16.4.1 POST /scenario/create

Create a new simulation scenario.

Request Body:

```
1 {
2   "name": "Add 3 to Review Team",
3   "resource_changes": {
4     "Review Team": 3
5   }
6 }
```

Response:

```
1 {
2   "scenario_id": "scenario_abc123",
3   "name": "Add 3 to Review Team",
4   "created_at": "2025-12-25T23:50:00Z"
5 }
```

16.4.2 POST /scenario/compare

Run and compare scenarios.

Request Body:

```
1 {
2   "scenario_id": "scenario_abc123",
3   "horizon_hours": 24,
4   "num_runs": 100
5 }
```

Response:

```
1 {
2   "baseline": {
3     "avg_throughput": 45.2,
4     "sla_compliance": 0.87,
5     "avg_cycle_time_hours": 18.5
6   },
7   "scenario": {
8     "avg_throughput": 52.8,
9     "sla_compliance": 0.94,
10    "avg_cycle_time_hours": 15.2
11  },
12  "improvement": {
13    "throughput_increase": 0.168,
14    "sla_improvement": 0.07,
15    "cycle_time_reduction": 3.3
16  }
17 }
```

17 Future Enhancements

17.1 Planned Features

- **Advanced Analytics:** Process mining algorithms (conformance checking, variant analysis)
- **Deep Learning Models:** LSTM-based sequence prediction
- **Real-time Dashboards:** WebSocket-based live updates
- **Multi-tenancy:** Support for multiple organizations
- **Advanced Simulation:** Agent-based modeling
- **Integration APIs:** Connectors for Appian, ServiceNow, Salesforce
- **Mobile App:** iOS/Android monitoring app
- **Automated Remediation:** Auto-scaling and resource reallocation

17.2 Research Directions

- Reinforcement learning for optimal resource allocation
- Causal inference for process improvement
- Federated learning for privacy-preserving analytics
- Graph neural networks for process discovery

18 Conclusion

The Appian SLA Predictive Simulation platform provides a comprehensive solution for process mining, predictive analytics, and simulation in business process management. By combining real-time event streaming, machine learning, and interactive visualization, the platform enables organizations to:

- Proactively manage SLAs and prevent breaches
- Identify and eliminate process bottlenecks
- Optimize resource allocation through simulation
- Make data-driven decisions based on predictive insights

This documentation provides the foundation for deploying, configuring, and extending the platform to meet specific organizational needs.

19 References

References

- [1] Apache Kafka Documentation, <https://kafka.apache.org/documentation/>
- [2] ClickHouse Documentation, <https://clickhouse.com/docs/>
- [3] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- [4] van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action*. Springer.
- [5] FastAPI Documentation, <https://fastapi.tiangolo.com/>
- [6] Docker Documentation, <https://docs.docker.com/>
- [7] Prometheus Documentation, <https://prometheus.io/docs/>
- [8] Grafana Documentation, <https://grafana.com/docs/>

A Appendix A: Sample Data

A.1 Sample Event Record

```
1 {  
2   "event_id": 12345,  
3   "case_id": "CASE_000123",  
4   "activity": "Manual Review",  
5   "status": "COMPLETED",  
6   "timestamp": "2025-12-25T14:30:00Z",  
7   "resource": "Agent_007",  
8   "sla_deadline": "2025-12-26T18:00:00Z",  
9   "priority": 2,  
10  "customer_tier": "Gold",  
11  "complexity_score": 7.5  
12 }
```

Listing 25: Sample Event JSON

B Appendix B: Configuration Files

B.1 Model Registry Example

```
1 models:  
2   - name: duration_predictor  
3     version: v1  
4     type: xgboost  
5     path: /app/weights/xgboost/v1/duration_model.json  
6     features:
```

```
7     - activity
8     - customer_tier
9     - complexity_score
10    - priority
11
12  - name: sla_breach_predictor
13    version: v1
14    type: xgboost
15    path: /app/weights/xgboost/v1/sla_model.json
16    features:
17      - activity
18      - hours_to_sla
19      - complexity_score
20      - customer_tier
```

Listing 26: registry.yaml

C Appendix C: Glossary

SLA Service Level Agreement - A commitment between a service provider and a client

WIP Work In Progress - Cases currently being processed

OLAP Online Analytical Processing - Database optimized for analytics

ML Machine Learning - Algorithms that learn from data

API Application Programming Interface - Interface for software interaction

REST Representational State Transfer - Architectural style for web services

JSON JavaScript Object Notation - Data interchange format

YAML YAML Ain't Markup Language - Human-readable data serialization

Docker Containerization platform for applications

Microservices Architectural pattern with independent services